



Data Structure CSE207

Md. Manowarul Islam
Guest Lecturer
East West University

Graph and Their Applications

Graph

- Nonlinear data Structures.
- A graph G consists of two properties:
 - (a) A set V of elements called vertices or nodes.
 - (b) A set E of connectors called edges such that each edge e is identified as $e = [u,v]$ (unordered pair of vertices). Here u and v are called endpoints of e and they are said to be the adjacent nodes or neighbors .
- The order of a graph is $|V|$ (the number of vertices).
- A graph's size is $|E|$, the number of edges.
- A tree is a graph with no cycle.

Example:

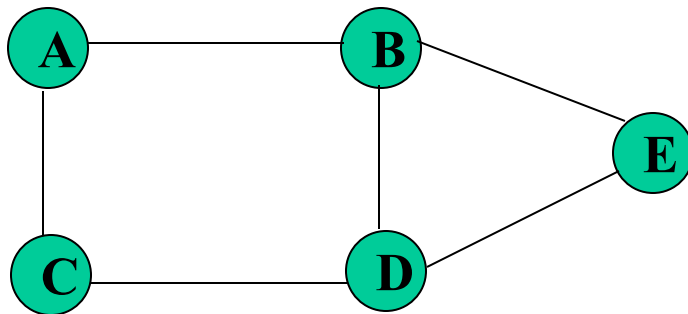
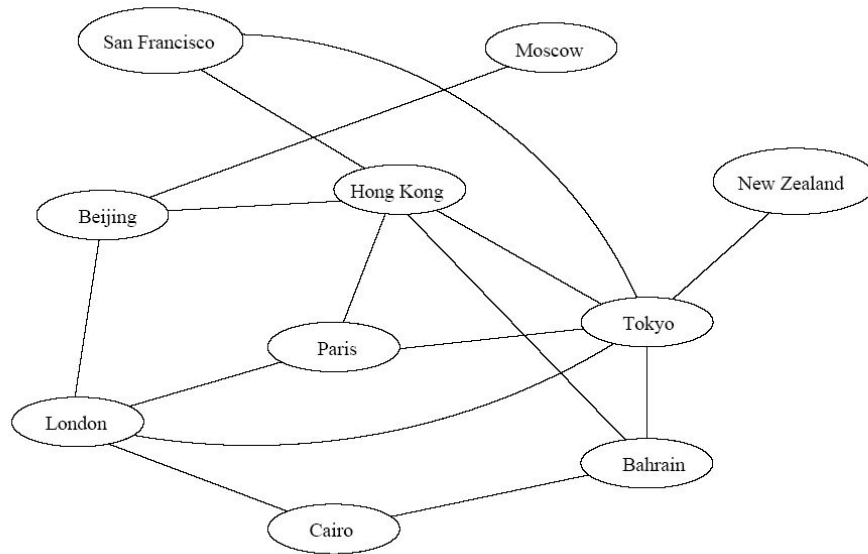


Figure1: A graph G_1

In Graph G_1

- 5 Vertices: $\{A, B, C, D, E\}$
- 6 Edges: $\{[A,B], [A,C], [B,D], [B,E], [C,D], [D,E]\}$

Application



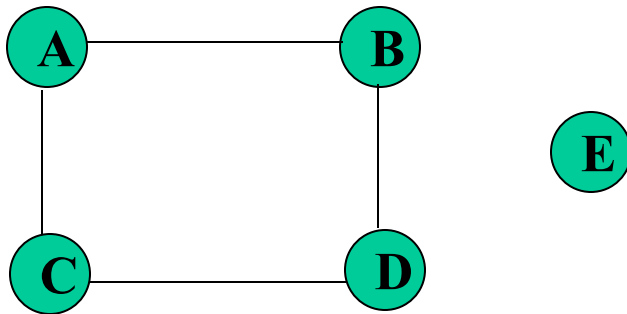
**Air flight
system**

- Each vertex represents a city
- Each edge represents a direct flight between two cities
- A query on **direct flights** = a query on whether an edge exists
- A query on **how to get to a location** = does a **path** exist from A to B
- We can even associate costs to **edges** (**weighted graphs**), then ask “what is the cheapest path from A to B”

Definitions

- **Isolated Node:** A vertex u with no edges.
- **Path:** A path P of length n from a node u to node v is defined as a sequence $n+1$ nodes such that $P=(v_0, v_1, \dots, v_n)$.
 - (a) Simple Path: if all nodes in path P are distinct.
 - (b) Closed Path: If node $v_0 = v_n$.
- **Cycle:** A closed path, with no other repeated vertices other than the starting and the ending vertices.

Example:



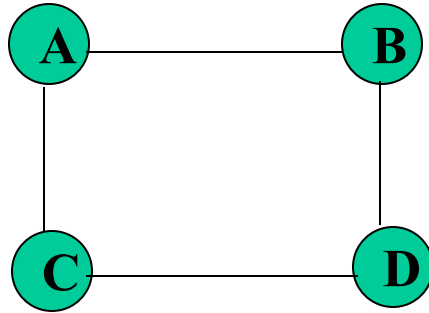
In graph G_2

- Isolate Node: E
- Path P : A, B, D, C
- Cycle C_1 : A, B, D, C, A

Figure2: Graph G_2

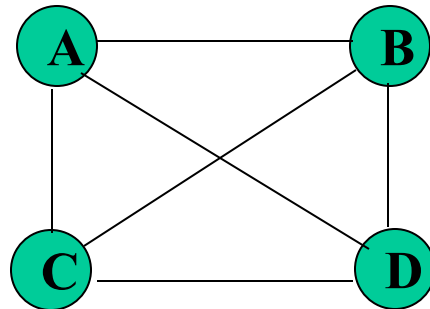
- **Connected Graph:** A graph is called connected if there is a simple path between any two of its nodes.

Example:



- **Complete Graph:** A graph G is complete if every node in graph G is adjacent to every other nodes. A complete graph with n nodes will have $n(n-1)/2$ edges.

Example:



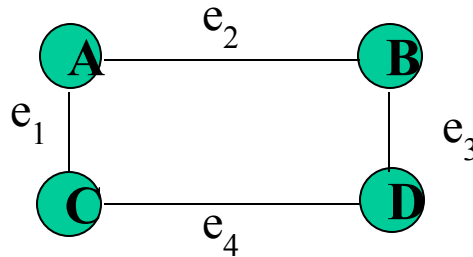
- **Tree Graph:** A connected graph with no cycle. If a tree graph has m nodes, then there are $m-1$ edges.

Example:



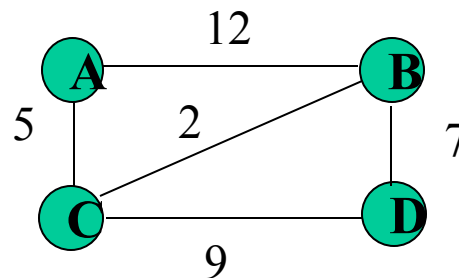
- **Labeled Graph:** A graph G is said to be labeled if its edges are assigned data.

Example:



- **Weighted Graph:** A labeled graph where each edge is assigned a nonnegative numerical value $w(e)$.

Example:

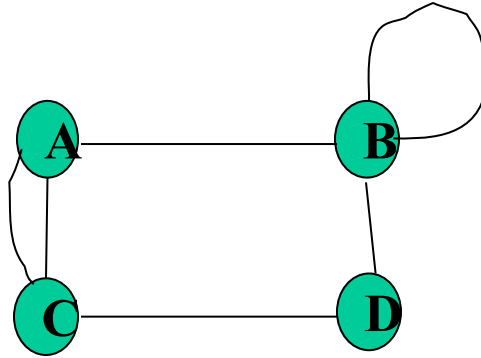


- **Multigraph:** A multigraph has the following properties:

(a) Multiple Edges

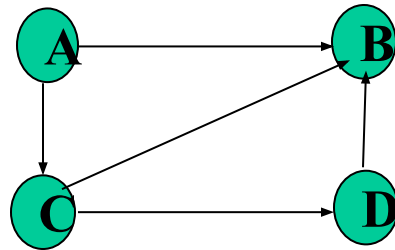
(b) Loops

Example:



- **Directed Graph:** Each edge in graph has a direction such that $e = (u,v)$, ie. e begins at u and ends at v .

Example:



• **Degree of a Node:** No. of edges connected to a node.

(a) **Indegree:** No. of edges ending at a node.

(b) **Outdegree:** No. of edges beginning at a node.

Example:

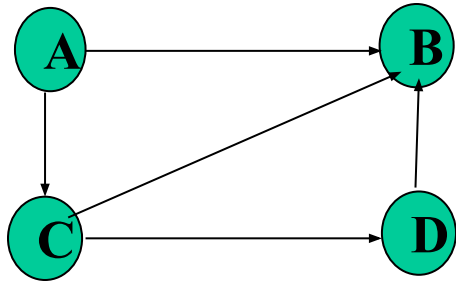


Figure 3: Directed Graph G

In graph G

Indeg(A)= 0 Outdeg(A)=2

Indeg(B)= 3 Outdeg(C)= 2

Indeg(C)= 1 Outdeg(B)= 0

Indeg(D)= 1 Outdeg(D)= 1

Note:

- A node u is called source if it has a positive outdegree and 0 indegree.
- A node u is called sink if it has a positive indegree and 0 outdegree.
- For a directed graph, a loop adds one to the indegree and one to the outdegree.
- For undirected graph, a loop adds two to the degree.

Representation of Graph

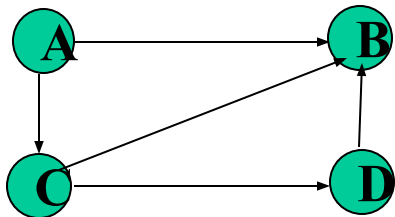
- (1) Sequential Representation
- (2) Linked Representation

Sequential Representation

- Use Adjacency Matrix (Boolean Matrix).
- An adjacency matrix $A = (a_{ij})$ of a graph G is the $m \times m$ matrix defined as follows:

$$a_{ij} = \begin{cases} 1 & \text{if } v_i \text{ is adjacent to } v_j \\ 0 & \text{otherwise} \end{cases}$$

Example:



	A	B	C	D
A	0	1	1	0
B	0	0	0	0
C	0	1	0	1
D	0	1	0	0

Figure 4: A Directed Graph & Its Adjacency Matrix

Graph Representation

- Two popular computer representations of a graph. Both represent the vertex set and the edge set, but in different ways.

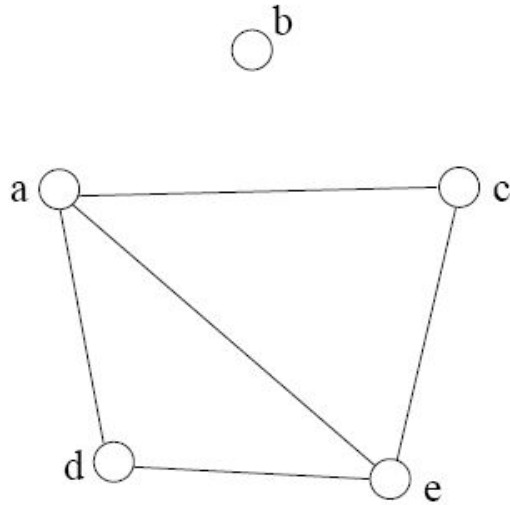
1. Adjacency Matrix

Use a 2D matrix to represent the graph

2. Adjacency List

Use a 1D array of linked lists

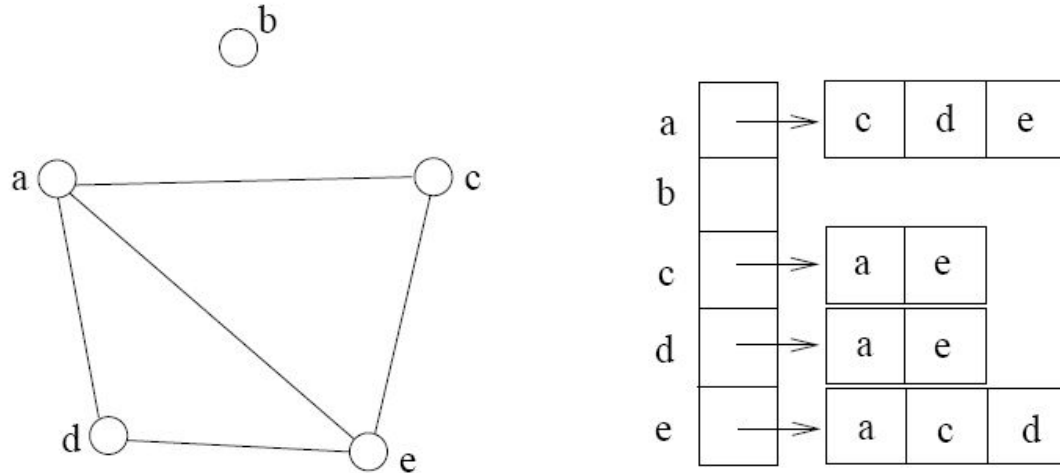
Adjacency Matrix



	a	b	c	d	e
a	0	0	1	1	1
b	0	0	0	0	0
c	1	0	0	0	1
d	1	0	0	0	1
e	1	0	1	1	0

- 2D array $A[0..n-1, 0..n-1]$, where n is the number of vertices in the graph
- Each row and column is indexed by the vertex id
 - e.g. $a=0, b=1, c=2, d=3, e=4$
- $A[i][j]=1$ if there is an edge connecting vertices i and j ; otherwise, $A[i][j]=0$

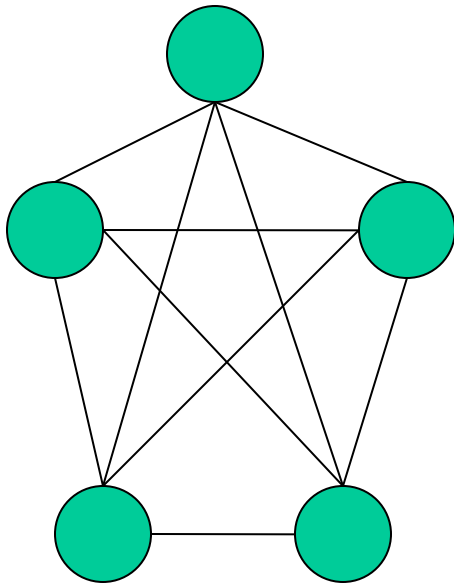
Adjacency List



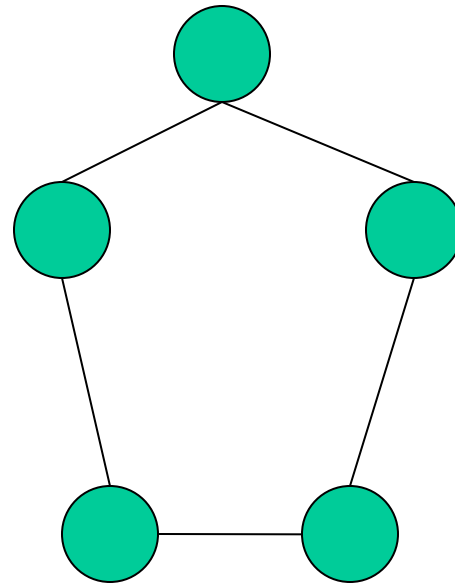
- If the graph is not dense, in other words, sparse, a better solution is an adjacency list
- The adjacency list is an array $A[0..n-1]$ of lists, where n is the number of vertices in the graph.
- Each array entry is indexed by the vertex id
- Each list $A[i]$ stores the ids of the vertices adjacent to vertex i

Which Representation Is Better?

- The adjacency-list is usually preferred if the graph is sparse (having few edges).
- The adjacency-matrix is preferred if the graph is dense (the number of edges is close to the maximal number of edges).



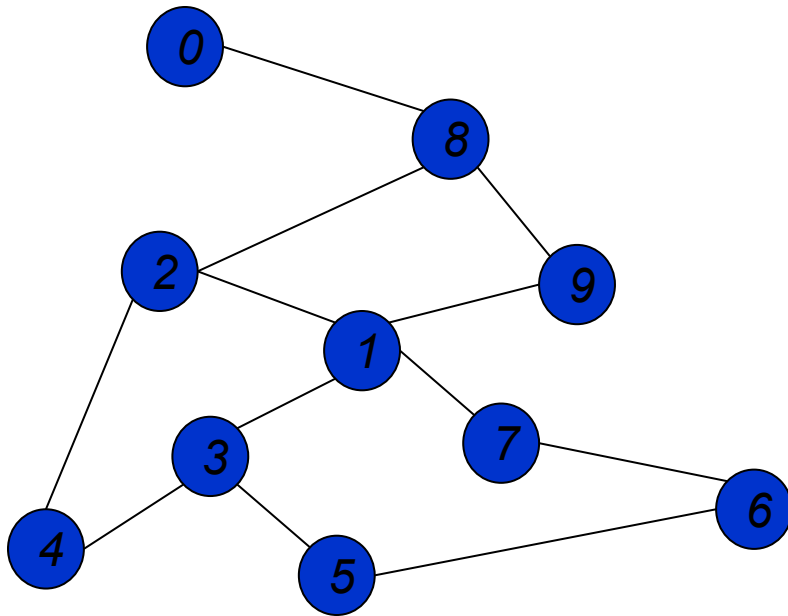
Dense Graph



Sparse Graph

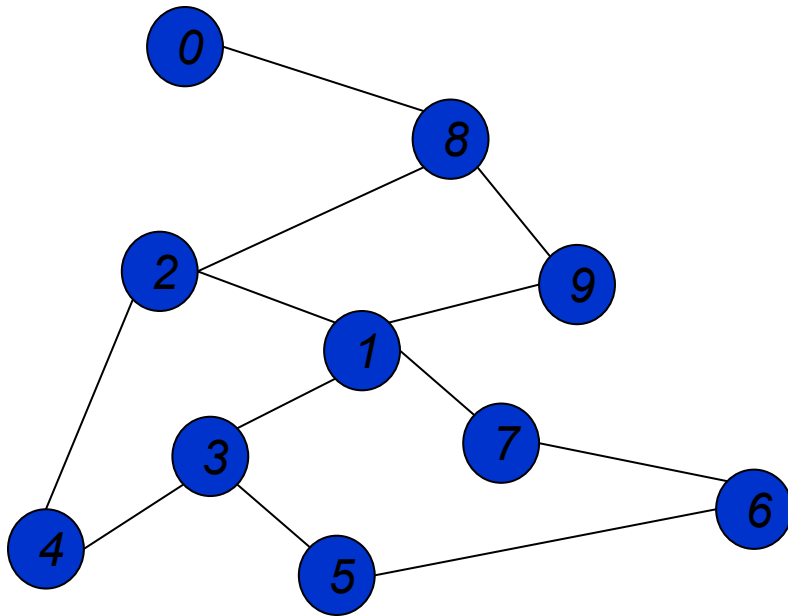
Figure: Dense and Sparse Graph

Adjacency Matrix Example



	0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	1	0
1	0	0	1	1	0	0	0	1	0	1
2	0	1	0	0	1	0	0	0	1	0
3	0	1	0	0	1	1	0	0	0	0
4	0	0	1	1	0	0	0	0	0	0
5	0	0	0	1	0	0	1	0	0	0
6	0	0	0	0	0	1	0	1	0	0
7	0	1	0	0	0	0	1	0	0	0
8	1	0	1	0	0	0	0	0	0	1
9	0	1	0	0	0	0	0	0	1	0

Adjacency List Example



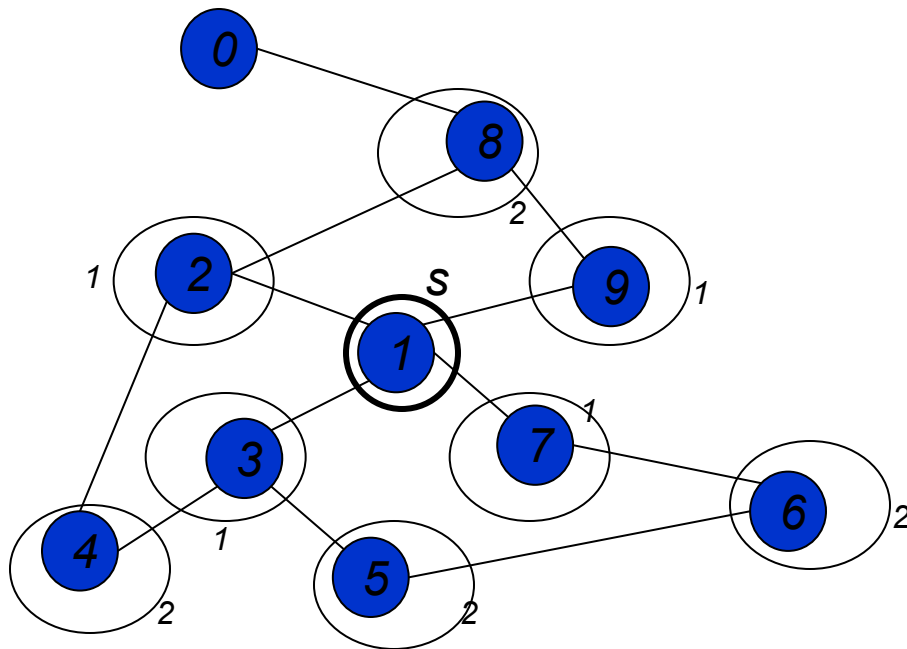
0	→	8
1	→	2 3 7 9
2	→	1 4 8
3	→	1 4 5
4	→	2 3
5	→	3 6
6	→	5 7
7	→	1 6
8	→	0 2 9
9	→	1 8

Graph Traversal

- Application example
 - Given a graph representation and a vertex s in the graph
 - Find paths from s to other vertices
- Two common graph traversal algorithms
 - Breadth-First Search (BFS)
 - ◆ Find the shortest paths in an unweighted graph
 - Depth-First Search (DFS)
 - ◆ Topological sort
 - ◆ Find strongly connected components

BFS and Shortest Path Problem

- Given any source vertex s , BFS visits the other vertices at **increasing distances** away from s . In doing so, BFS discovers paths from s to other vertices
- What do we mean by “distance”? The number of edges on a path from s



Example

Consider s =vertex 1

Nodes at distance 1?

2, 3, 7, 9

Nodes at distance 2?

8, 6, 5, 4

Nodes at distance 3?

0

Graph Searching

- Given: a graph $G = (V, E)$, directed or undirected
- Goal: methodically explore every vertex and every edge
- Ultimately: build a tree on the graph
 - Pick a vertex as the root
 - Choose certain edges to produce a tree
 - Note: might also build a *forest* if graph is not connected

Breadth-First Search

- “Explore” a graph, turning it into a **tree**
 - One vertex at a time
 - Expand frontier of explored vertices across the *breadth* of the frontier
- Builds a tree over the graph
 - Pick a *source vertex* to be the root
 - Find (“discover”) its children, then their children, etc.

Breadth-First Search

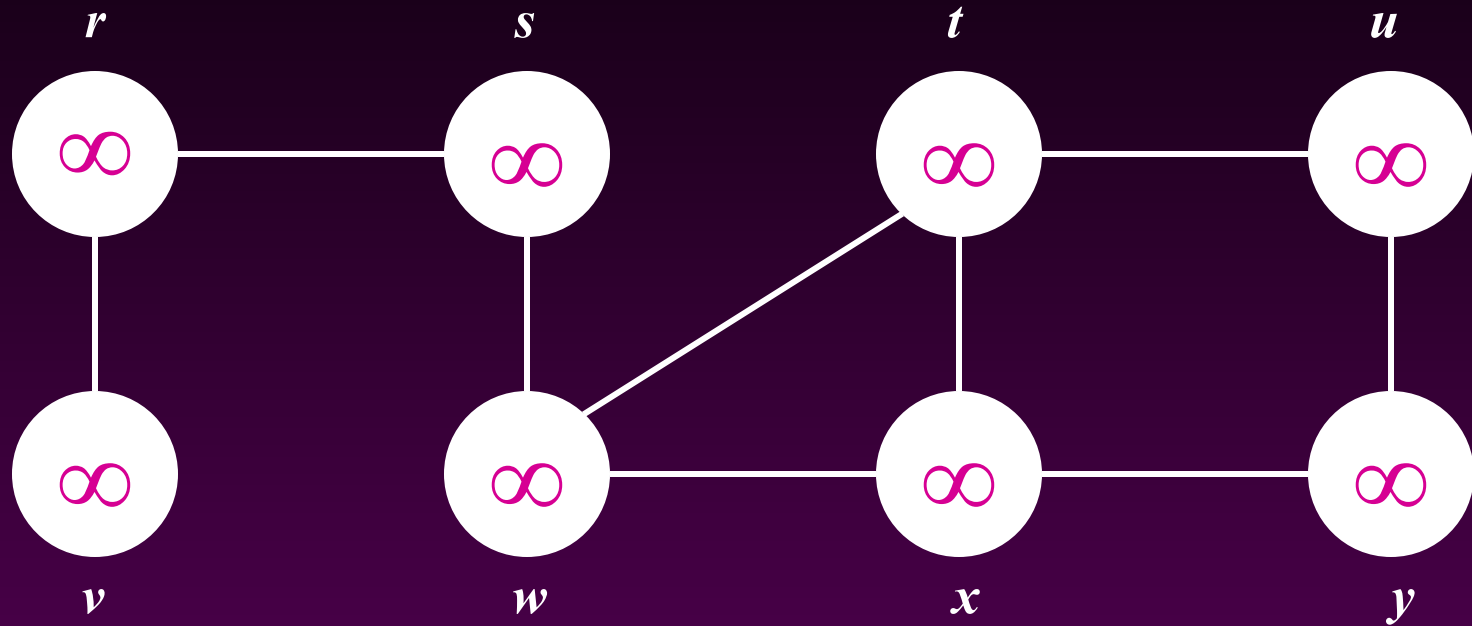
- Every vertex of a graph contains a color at every moment:
 - **White vertices** have not been discovered
 - All vertices start with white initially
 - **Grey vertices** are discovered but not fully explored
 - They may be adjacent to white vertices
 - **Black vertices** are discovered and fully explored
 - They are adjacent only to black and gray vertices
- Explore vertices by scanning adjacency list of grey vertices

Breadth-First Search

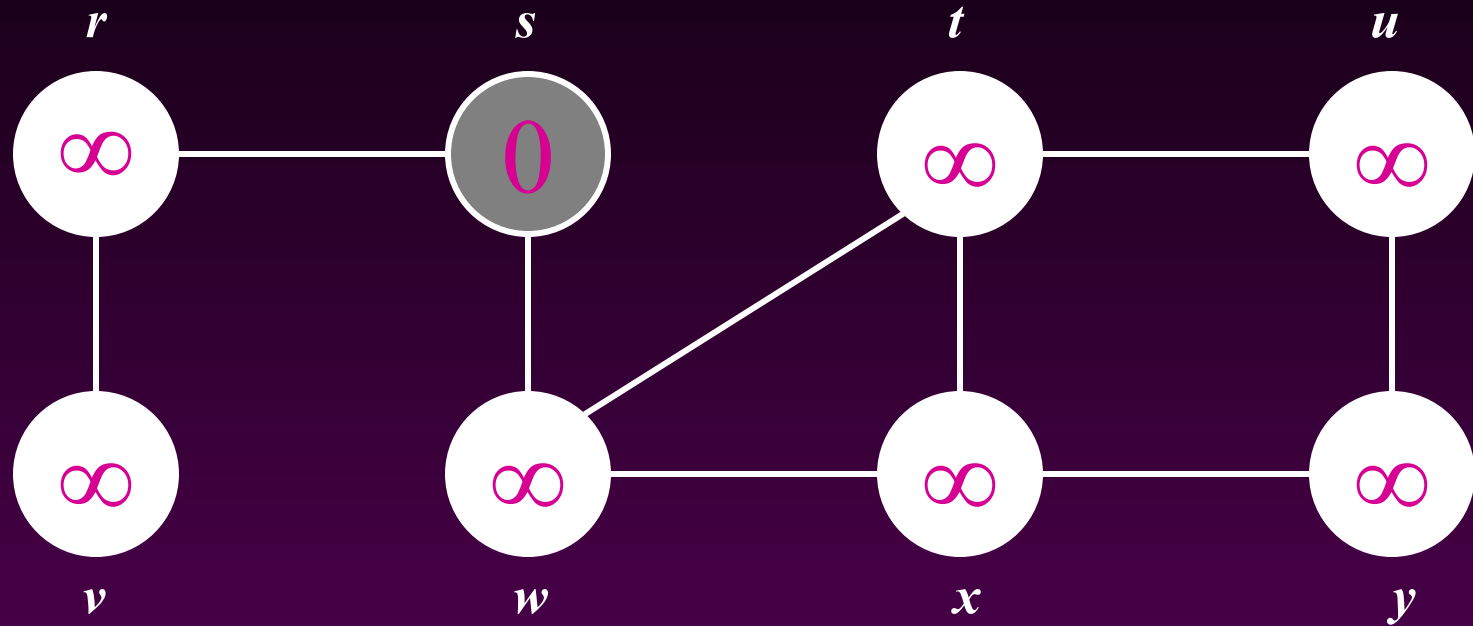
```
BFS(G, s) {  
    initialize vertices;  
    Q = {s};    // Q is a queue (duh); initialize to s  
    while (Q not empty) {  
        u = RemoveTop(Q);  
        for each v ∈ u->adj {  
            if (v->color == WHITE)  
                v->color = GREY;  
                v->d = u->d + 1;  
                v->p = u;  
                Enqueue(Q, v);  
        }  
        u->color = BLACK;  
    }  
}
```

**What does $v \rightarrow d$
represent?
What does $v \rightarrow p$
represent?**

Breadth-First Search: Example

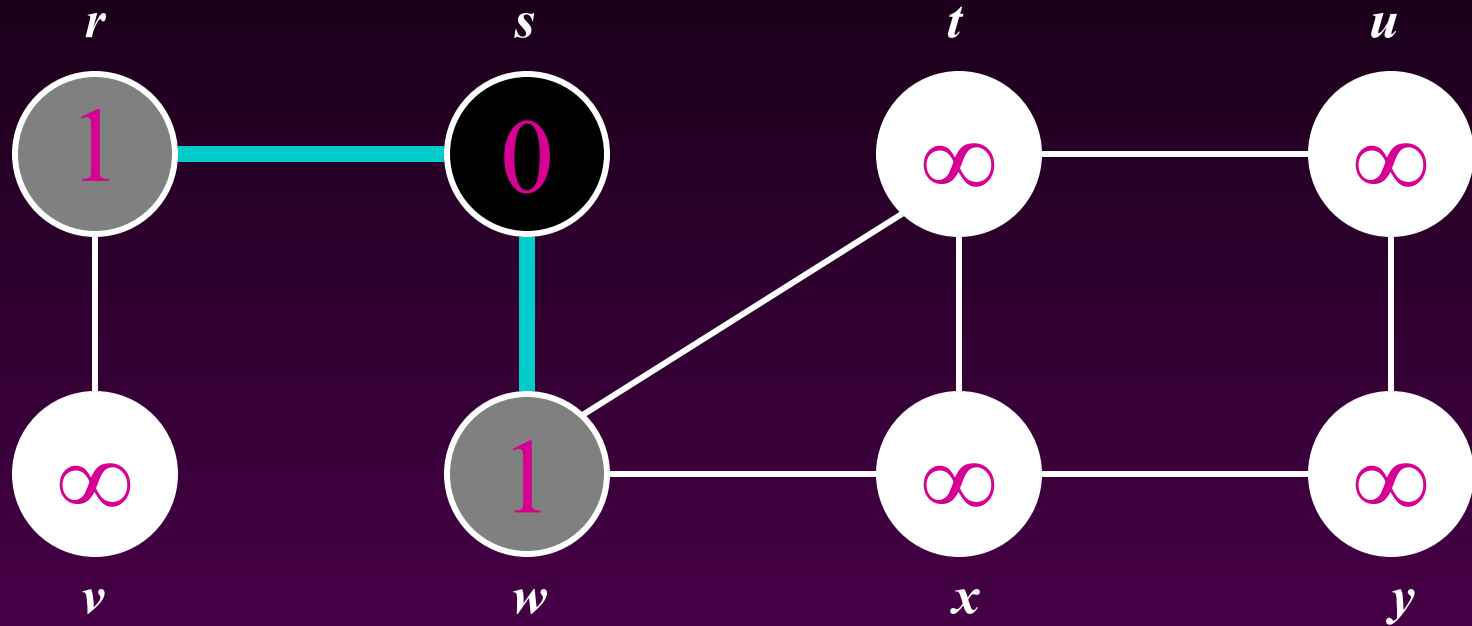


Breadth-First Search: Example



Q : s

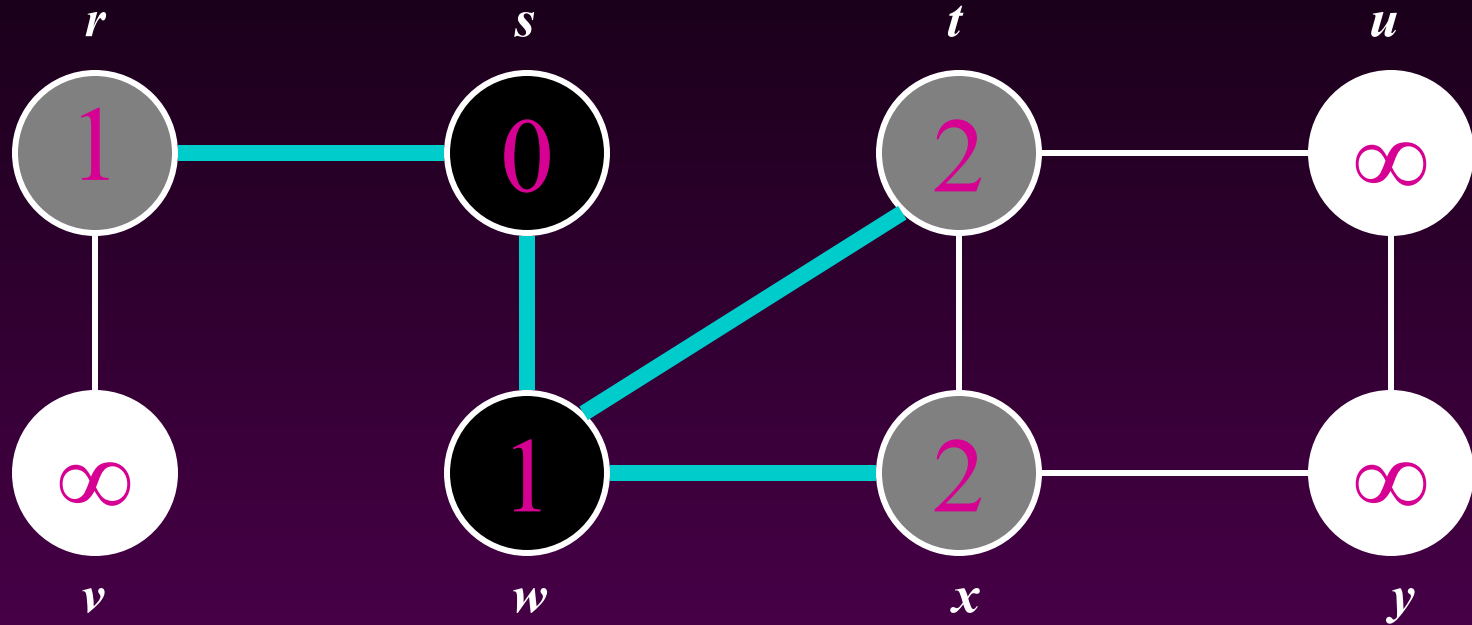
Breadth-First Search: Example



Q :

w	r
-----	-----

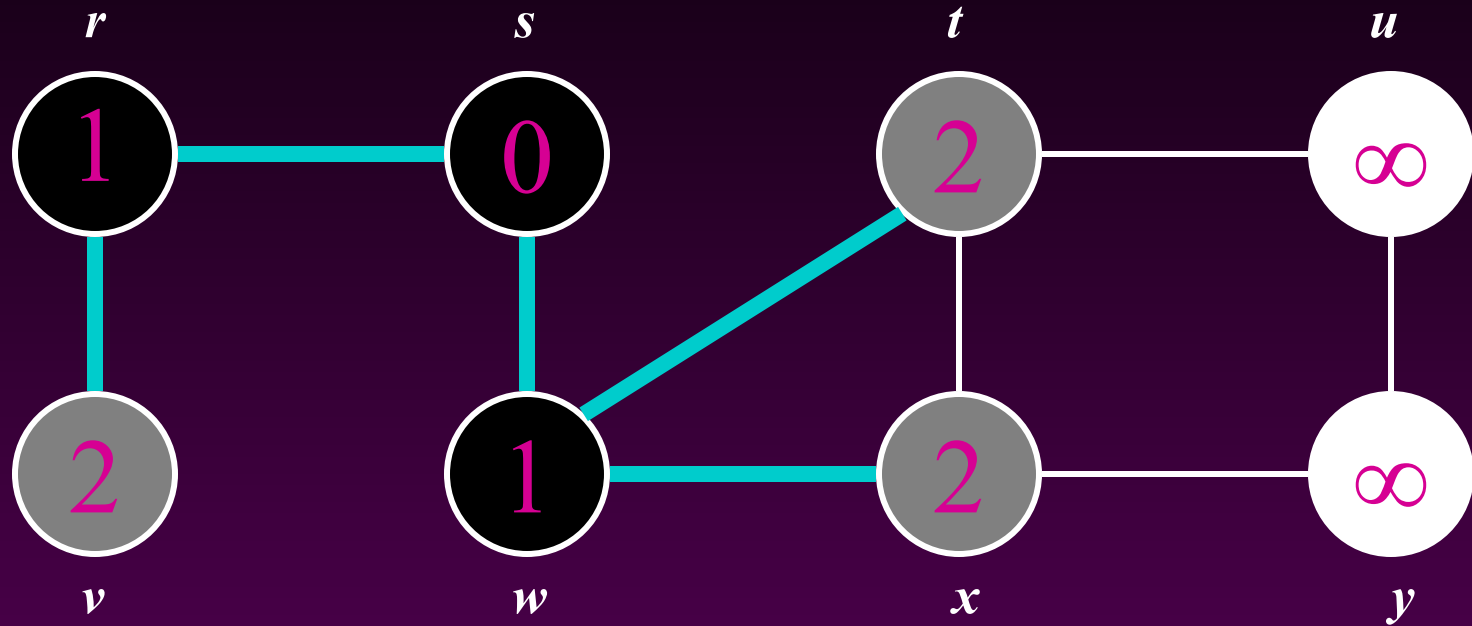
Breadth-First Search: Example



Q :

r	t	x
-----	-----	-----

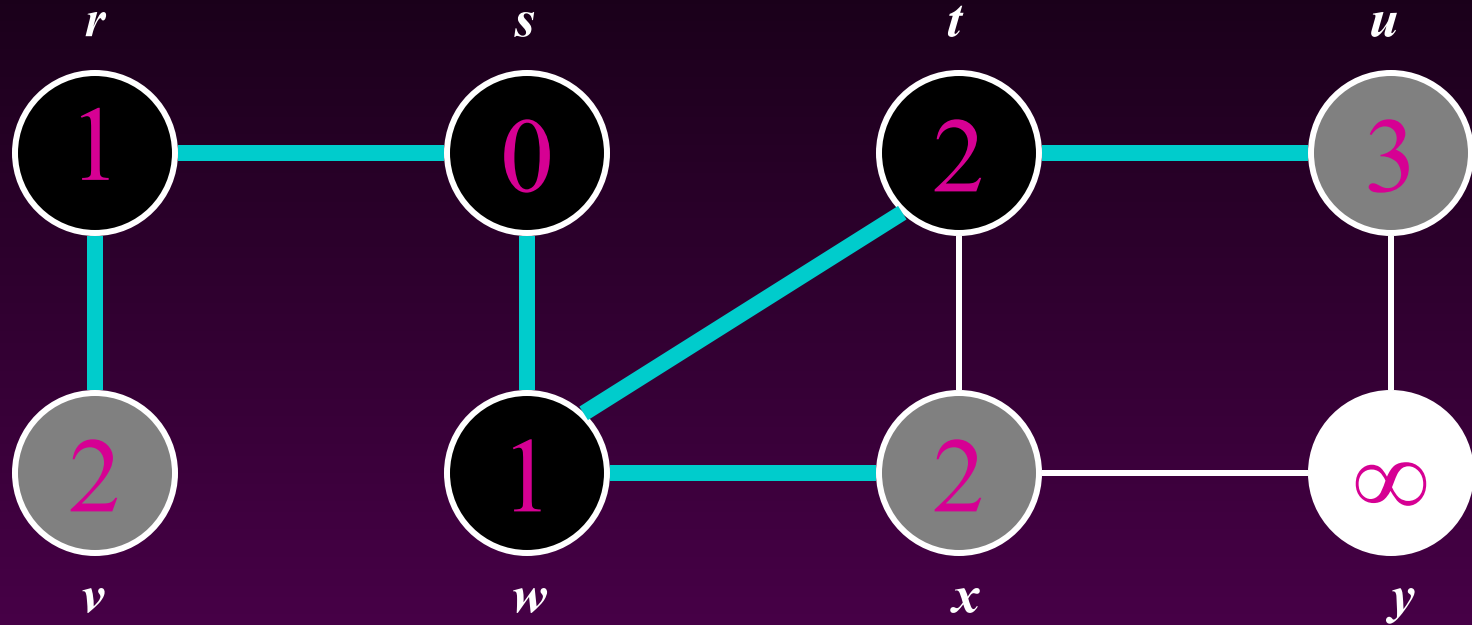
Breadth-First Search: Example



Q :

t	x	v
-----	-----	-----

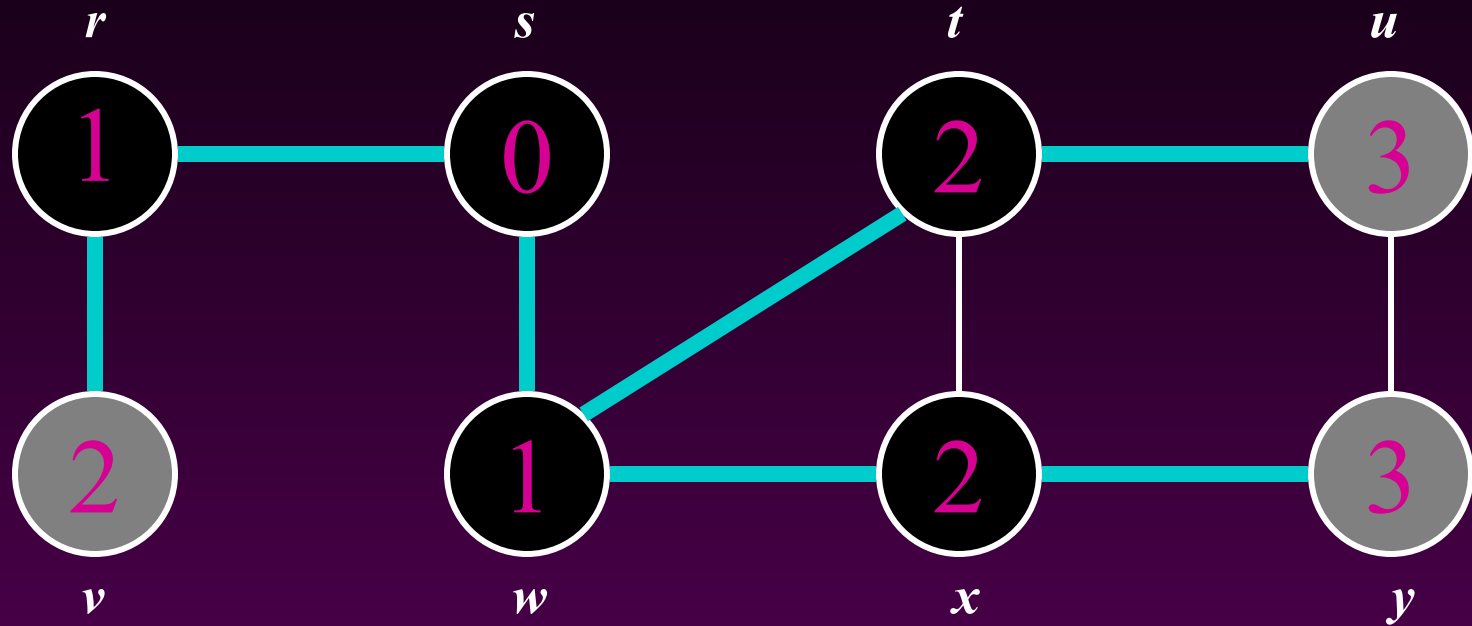
Breadth-First Search: Example



Q :

x	v	u
-----	-----	-----

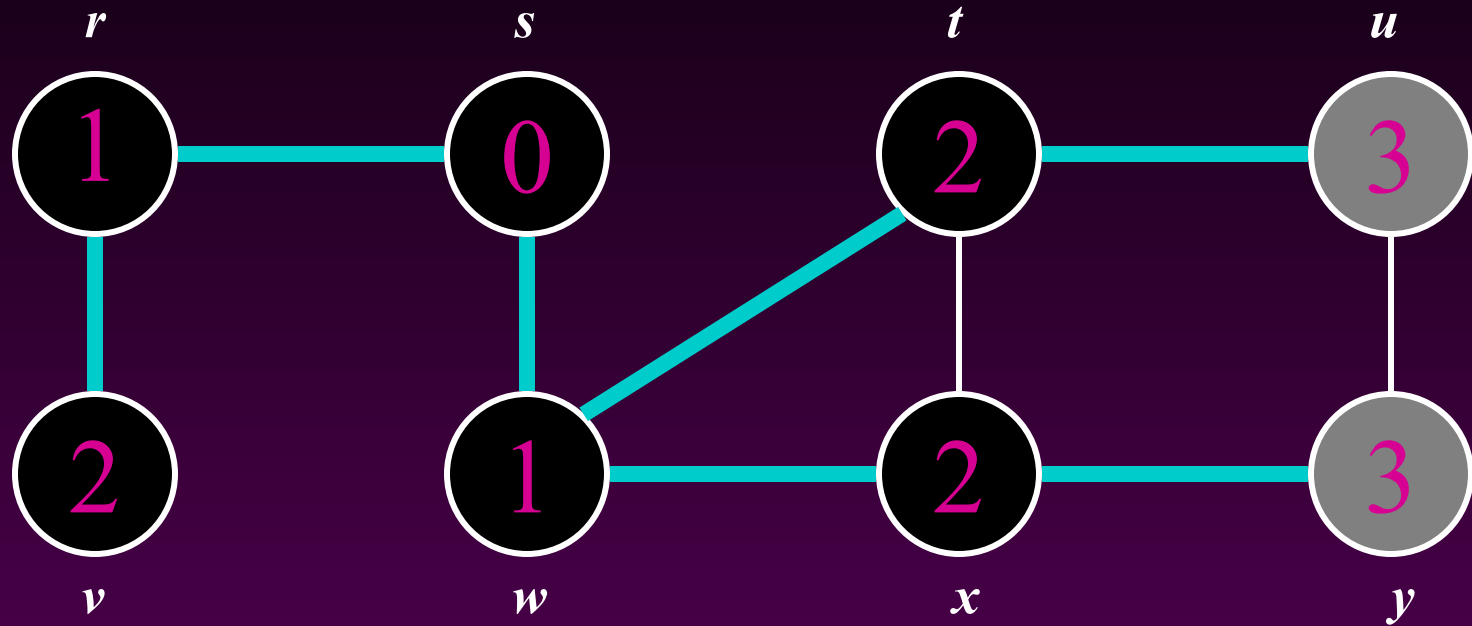
Breadth-First Search: Example



Q :

v	u	y
-----	-----	-----

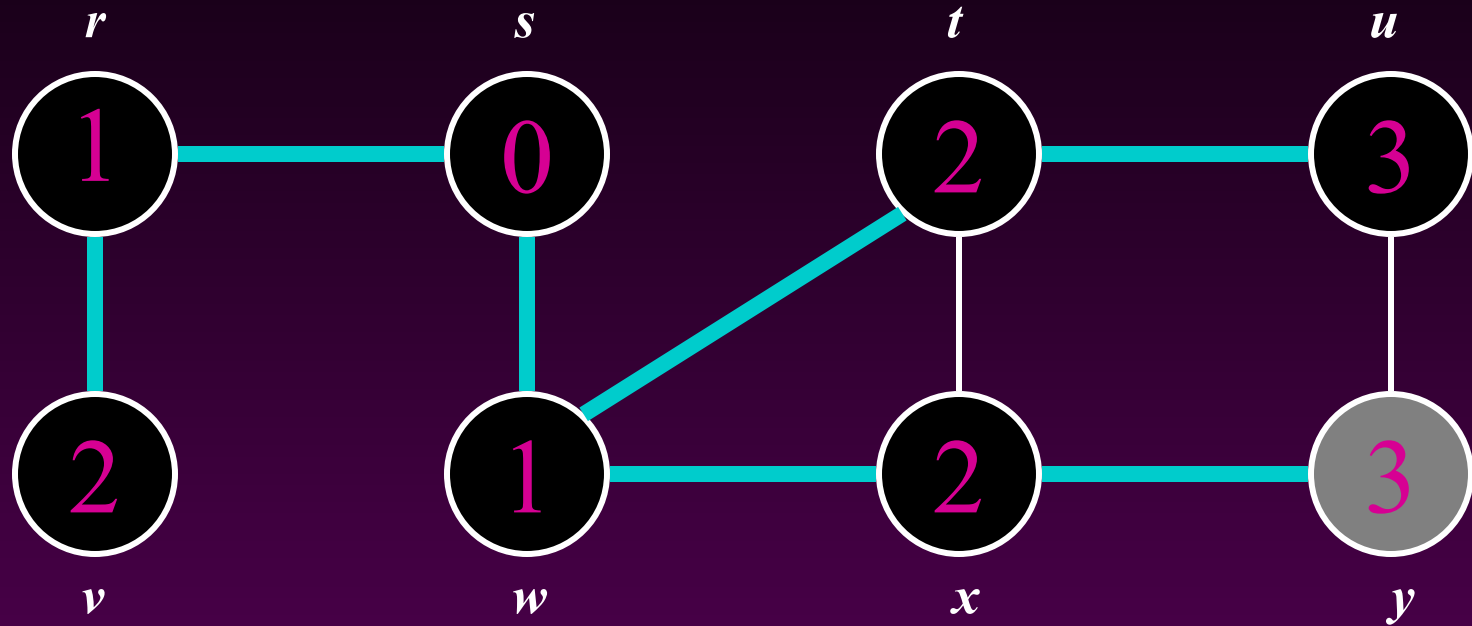
Breadth-First Search: Example



Q :

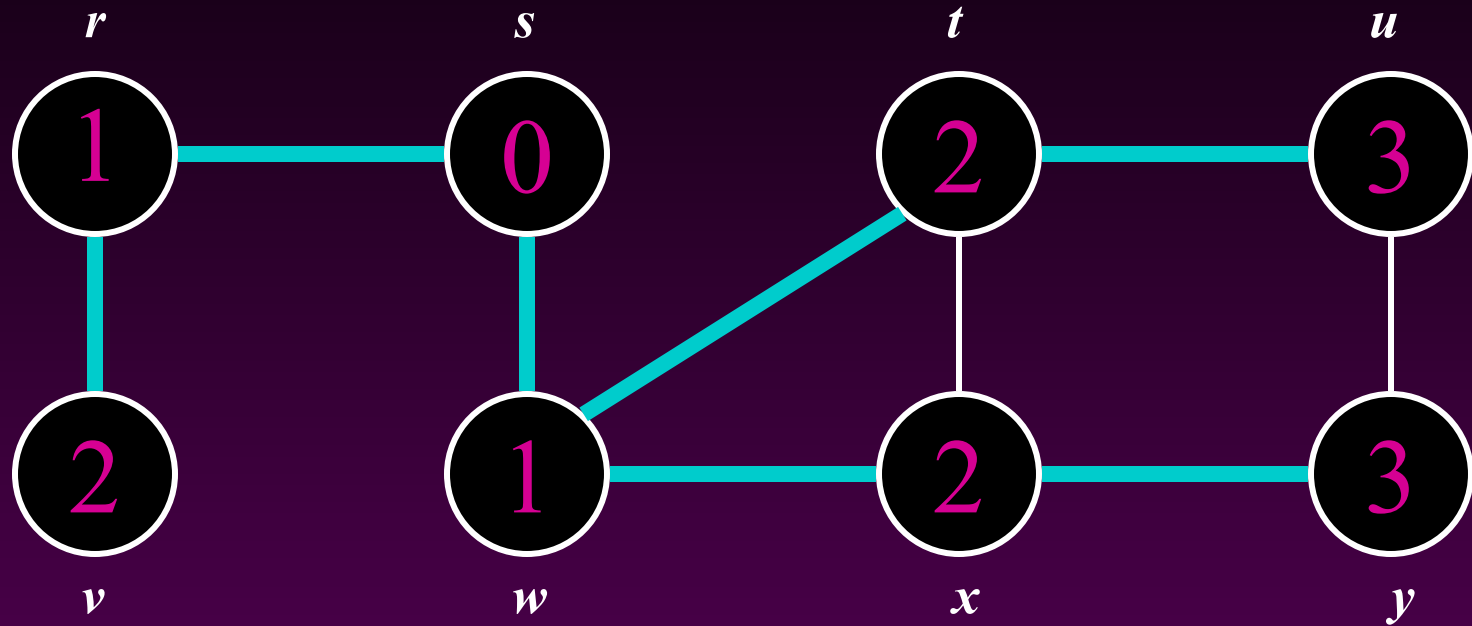
u	y
-----	-----

Breadth-First Search: Example



Q : y

Breadth-First Search: Example



$Q: \emptyset$

Breadth-First Search: Properties

- BFS calculates the *shortest-path distance* to the source node
 - Shortest-path distance $\delta(s,v)$ = minimum number of edges from s to v , or ∞ if v not reachable from s
 - Proof given in the book (p. 472-5)
- BFS builds *breadth-first tree*, in which paths to root represent shortest paths in G
 - Thus can use BFS to calculate shortest path from one vertex to another in $O(V+E)$ time

Breadth-First Search: The Code

Data: `color[V], prev[V], d[V]`

```
BFS(G) // starts from here
{
    for each vertex  $u \in V - \{s\}$ 
    {
        color[u]=WHITE;
        prev[u]=NIL;
        d[u]=inf;
    }
    color[s]=GRAY;
    d[s]=0; prev[s]=NIL;
    Q=empty;
    ENQUEUE(Q, s);
```

```
While(Q not empty)
{
    u = DEQUEUE(Q);
    for each  $v \in \text{adj}[u]$ 
    {
        if (color[v] == WHITE) {
            color[v] = GRAY;
            d[v] = d[u] + 1;
            prev[v] = u;
            Enqueue(Q, v);
        }
    }
    color[u] = BLACK;
}
}
```

Breadth-First Search: Print Path

Data: color[V], prev[V], d[V]

```
Print-Path(G, s, v)
{
    if (v==s)
        print(s)
    else if (prev[v]==NIL)
        print(No path);
    else{
        Print-Path(G, s, prev[v]);
        print(v);
    }
}
```

BFS – Questions

- Find the shortest path between “A” and “B” (with path)? When will it fail?
- Find the most distant node from start node “A”
- How can we detect that there exists no path between A and B using BFS?
- Print all of those nodes that are at distance 2 from source vertex “S”.
- How can we modify BFS algorithm to check the bipartiteness of a graph?
- Is it possible to answer that there exists more than one path from “S” to “T” with minimum path cost?